0.12 DevSecOps

Topic 12 of 13 | ~3 hours | Difficulty: Beginner

1 What it is

DevSecOps means building security into every stage of the delivery lifecycle — design, code, build, test, deploy, run — as a shared responsibility of developers, operators and security engineers, instead of a separate inspection performed by a security team just before release. The core slogan is **shift left**: find and fix security problems as early (as far "left" on the timeline from idea to production) as possible, because a vulnerability found in a pull request costs minutes to fix and one found in production costs incident response, customer notification and reputation.

Security checks become **automated steps in the pipeline** (CI/CD §0.2), run on every change: scanning code for flaws (**SAST**, static application security testing), scanning third-party libraries for known vulnerabilities (**SCA**, software composition analysis), scanning for committed secrets, scanning container images and infrastructure code, and testing the running application (**DAST**, dynamic application security testing). The newest strand is **software supply chain security**: knowing and protecting everything that goes into your software — your dependencies, your build system, your base images — and proving where each artifact came from.

2 Why it matters in DevOps

- **Speed breaks manual security.** A team deploying 20 times a day cannot wait for a two-week security review. Automation lets security keep pace.
- **Most of your code is not yours.** A typical application is mostly open-source dependencies and a base image. Their vulnerabilities are yours the day you ship them.
- **Pipelines are privileged.** CI/CD systems hold secrets and can deploy to production, so an attacker who compromises the pipeline owns everything it can reach.
- **Misconfiguration is as dangerous as bugs.** Public storage buckets, open security groups and root containers are configuration errors that infrastructure as code lets you review and scan (Infrastructure as Code §0.3).

- **Compliance and customers demand evidence.** Audits and enterprise buyers increasingly ask for scan results, SBOMs and access controls.

3 Real-world system example

Log4Shell (December 2021). A critical remote-code-execution vulnerability (CVE-2021-44228) in Apache Log4j, a ubiquitous Java logging library, forced organisations worldwide to answer one question: "where do we use Log4j?" Teams with an inventory of their dependencies (an **SBOM, software bill of materials**) and automated scanning could answer within hours; teams without one searched servers by hand for days. The incident is a standard case for why dependency inventory and scanning belong in the pipeline.

SolarWinds (2020). Attackers compromised the build system of the SolarWinds Orion software and inserted malicious code into updates that were then signed and distributed to customers. It is the canonical example of a **supply chain attack**: the source repository looked normal, the compromise lived in the build. This drove interest in hardened, reproducible builds and provenance (the SLSA framework, now an OpenSSF project; Sigstore/cosign for signing).

GitHub. GitHub offers, as part of the platform, several of the controls described here: **Dependabot** (automated dependency update pull requests and alerts), **secret scanning** with push protection that blocks commits containing known token formats, and **code scanning** with CodeQL. See Level 11: Dependency Scanning §11.10 and Level 11: Secrets Scanning §11.11.

Kubernetes and containers. Open-source scanners such as Trivy (container images, filesystems, IaC, SBOMs) and Grype, and policy engines such as OPA Gatekeeper and Kyverno, are widely used to block vulnerable or non-compliant workloads before they run. See Level 11: Container Image Scanning §11.9.

4 Diagram

Security checks along the pipeline, each one cheap and automatic:

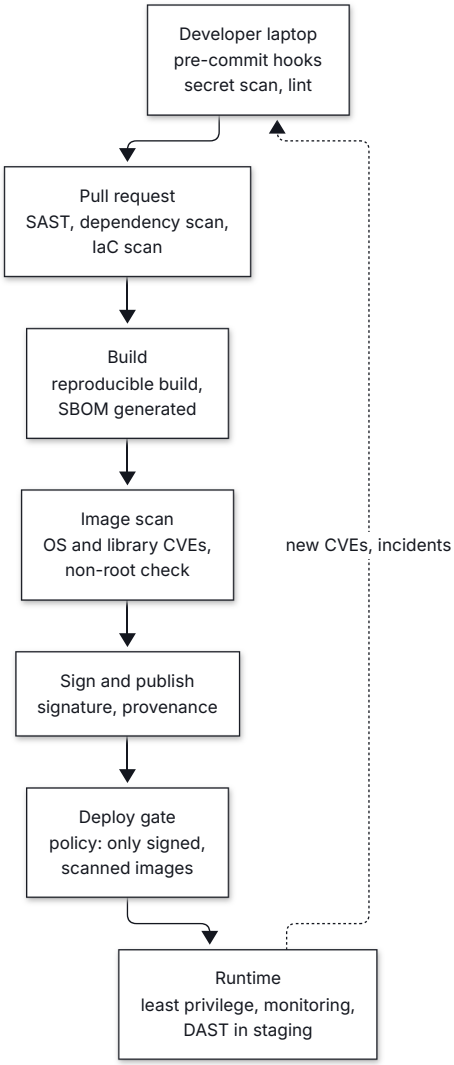


Figure 0.12-1

The supply chain you are responsible for:

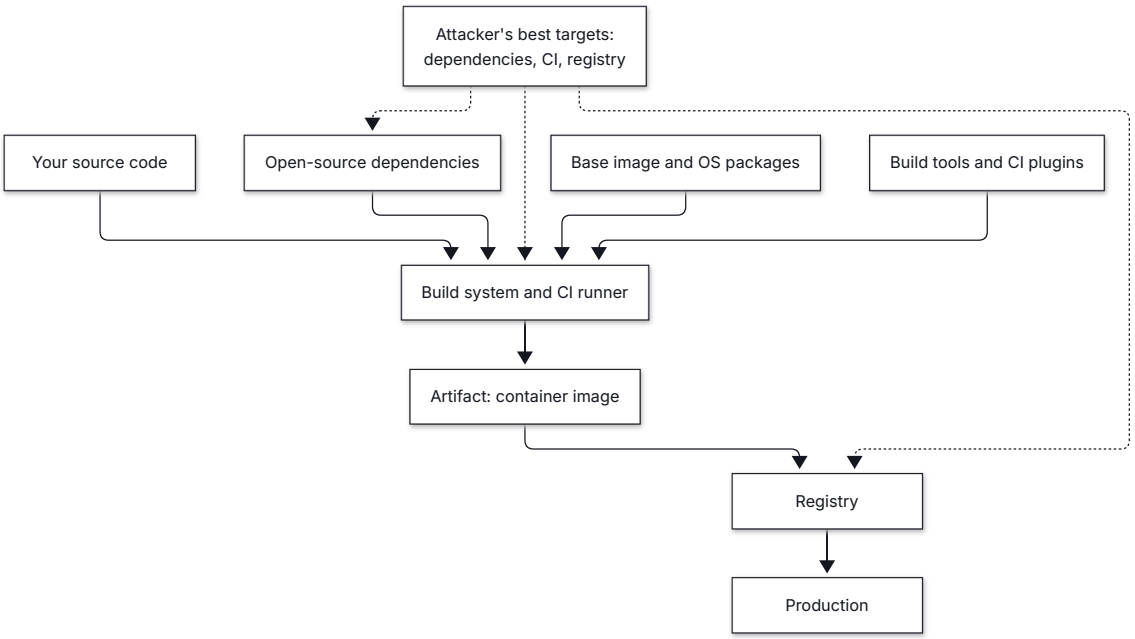


Figure 0.12-2

5 How it works

The security checks and where they run

Check	Looks at	Catches	Tools (examples)	Typical stage
Secret scanning	Commits, files	Committed API keys, passwords, private keys	gitleaks, GitHub secret scanning, truffleHog	pre-commit and PR
SAST	Your source code	Injection, unsafe functions, insecure patterns	Semgrep, CodeQL, SonarQube	PR
SCA / dependency scan	Lockfiles, manifests	Known CVEs in libraries, bad licenses	Dependabot, Snyk, `npm audit`, OWASP Dependency-Check	PR and nightly
IaC scan	Terraform, Kubernetes YAML	Open ports, public buckets, root containers	Checkov, tfsec/Trivy, kube-linter	PR
Image scan	Container images	OS and library CVEs, misconfig	Trivy, Grype	Build and registry
DAST	Running app	XSS, auth flaws, headers, exposed endpoints	OWASP ZAP	Staging

Check	Looks at	Catches	Tools (examples)	Typical stage
Policy / admission	Anything about to be deployed	Unsigned or non-compliant artifacts	OPA, Kyverno, cosign verification	Deploy

Principles that make it work

1. **Shift left, but keep defence in depth.** Early checks are cheap; later ones catch what slipped through. You need both.
2. **Fail the build on what matters, not on everything.** Block on critical, exploitable findings; report the rest. A gate that fails constantly gets disabled or ignored.
3. **Make the secure way the easy way.** Provide hardened base images, pipeline templates and secrets tooling (Platform Engineering §0.13) so developers do not need to be security experts.
4. **Least privilege everywhere.** Pipelines, humans and services each get the minimum access needed (Level 11: Least-privilege IAM §11.2).
5. **Treat secrets as toxic.** Never in Git or images; injected at runtime from a secrets manager (Level 11: Secrets Management §11.4).
6. **Know what you ship.** Generate an SBOM, pin versions and digests, keep a base-image update cadence.
7. **Verify provenance.** Sign artifacts, record how they were built, and let the deploy step verify signatures.
8. **Shared responsibility.** Security engineers build the guardrails and the knowledge; development teams own the fixes for their services.

Pinning, digests and integrity

Dependency and image tags can be moved. A version `==1.2.3` is better than `>=1.0`, and an image **digest** (`@sha256:...`) is better than a tag, because a digest names exact content. Checksums and signatures let a consumer prove that what they downloaded is what the publisher produced.

6 Step-by-step guide

Goal: build a toy "security gate" that scans a small repo for committed secrets, unpinned dependencies and a root container, install it as a **pre-commit hook** (shift-left on the developer's laptop), and then verify an artifact's integrity with a checksum (the supply chain idea in miniature). **Prerequisites:** Bash, `grep`, `sha256sum` (macOS: `shasum -a 256`) and Git. All work happens in a throwaway directory. The example keys are the documentation placeholders,

not real credentials. **Cost: none.** In real projects you would use maintained tools (gitleaks, Trivy, Semgrep, Dependabot — covered in Level 11); the toy gate exists so you understand *what* they do and *where* they run.

1. **Create a sample repo with deliberate problems.**

```bash
mkdir -p ~/devsecops-demo/repo && cd ~/devsecops-demo
cat > repo/app.py <<'EOF'
import os
AWS_KEY = "AKIAIOSFODNN7EXAMPLE"
db_password = "hunter2hunter2"
def handler(): return "ok"
EOF
printf 'flask\nrequests>=2.0\ngunicorn==22.0.0\n' > repo/requirements.txt
printf 'FROM python:latest\nCOPY . /app\nCMD ["python","/app/app.py"]\n' > repo/Dockerfile
```

2. **Write the gate.** It runs three checks and exits non-zero if any fails.

```bash
cat > gate.sh <<'EOF'
#!/usr/bin/env bash
# gate.sh - a toy "shift-left" security gate. Run it before you commit and again in CI.
# Usage: ./gate.sh <repo-dir>      exit code 0 = pass, 1 = fail
set -uo pipefail
REPO="${1:?usage: gate.sh <repo-dir>}"
FAIL=0
fail() { echo "  FAIL: $1"; FAIL=1; }

echo "[1/4] secret scan"
# AWS-style access key IDs, private key headers, and hard-coded password assignments
if grep -rEn --exclude-dir=.git 'AKIA[0-9A-Z]{16}|-----BEGIN (RSA |EC )?PRIVATE KEY-----|
  (password|secret|token)[[:space:]]*=[[:space:]]*["'"'"'][^"'"'"']{6,}' "$REPO" ; then
  fail "possible secret committed (see lines above)"
fi

echo "[2/4] dependency pinning"
for f in "$REPO"/requirements.txt; do
  [ -f "$f" ] || continue
  if grep -Evn '^\s*(#|$)|==' "$f"; then fail "unpinned dependency in $f (use
  name==version)"; fi
done

echo "[3/4] Dockerfile hygiene"
D="$REPO/Dockerfile"
if [ -f "$D" ]; then
  grep -Eqn '^FROM .*:latest|^FROM [^: ]+( |$)' "$D" && fail "base image not pinned to a
  version (avoid :latest or no tag)"
  grep -Eq '^USER ' "$D" || fail "container runs as root (add a USER instruction)"
fi
```

```
echo "[4/4] result"
[ "$FAIL" -eq 0 ] && echo "  PASS: gate clean" || echo "  GATE FAILED"
exit "$FAIL"
EOF
chmod +x gate.sh
```

3. **Run it against the bad repo.**

```bash
./gate.sh repo; echo "exit=$?"
```

Expected:

```
[1/4] secret scan
repo/app.py:2:AWS_KEY = "AKIAIOSFODNN7EXAMPLE"
repo/app.py:3:db_password = "hunter2hunter2"
  FAIL: possible secret committed (see lines above)
[2/4] dependency pinning
1:flask
2:requests⩾2.0
  FAIL: unpinned dependency in repo/requirements.txt (use name==version)
[3/4] Dockerfile hygiene
  FAIL: base image not pinned to a version (avoid :latest or no tag)
  FAIL: container runs as root (add a USER instruction)
[4/4] result
  GATE FAILED
exit=1
```

In a pipeline this non-zero exit code is what turns a pull request red.

4. **Fix the problems the secure way:** secrets come from the environment (injected from a secrets manager at runtime), dependencies are pinned, the image tag is explicit, and the process runs as a non-root user.

```bash
cat > repo/app.py <<'EOF'
import os
AWS_KEY = os.environ["AWS_ACCESS_KEY_ID"]
db_password = os.environ["DB_PASSWORD"]
def handler(): return "ok"
EOF
printf 'flask==3.0.3\nrequests==2.32.3\ngunicorn==22.0.0\n' > repo/requirements.txt
printf 'FROM python:3.12-slim\nCOPY . /app\nUSER 10001\nCMD ["python","/app/app.py"]\n' >
  repo/Dockerfile
./gate.sh repo; echo "exit=$?"
```

Expected: all four stages with no `FAIL` lines, `PASS: gate clean`, `exit=0`.

5. **Shift it left: install it as a Git pre-commit hook** in a fresh repository, so a secret never even reaches a commit.

```bash
mkdir hookdemo && cd hookdemo && git init -q .
cp ../gate.sh .
cat > .git/hooks/pre-commit <<'EOF'
#!/usr/bin/env bash
./gate.sh . || { echo "commit blocked by security gate"; exit 1; }
EOF
chmod +x .git/hooks/pre-commit
echo 'api_token = "abcd1234efgh"' > config.py
git add config.py && git commit -m "add config"; echo "exit=$?"
```

Expected: the gate prints `FAIL: possible secret committed`, followed by `commit blocked by security gate` and `exit=1`. The secret never entered history. (If `git commit` complains about your identity, run `git config user.name demo` and `git config user.email demo@example.com` inside this throwaway repo.) Now do it correctly:

```bash
echo 'api_token = os.environ["API_TOKEN"]' > config.py
git add config.py && git commit -q -m "add config"; echo "exit=$?"
```

Expected: the gate passes and the commit succeeds (`exit=0`). Local hooks can be bypassed (`--no-verify`), which is why the same gate must also run in CI where it cannot be skipped.

6. **Supply chain in miniature: integrity of an artifact.** A publisher records a checksum; a consumer verifies before using it.

```bash
cd ~/devsecops-demo && mkdir sc && cd sc
echo "release v1" > shop-api.tar
sha256sum shop-api.tar > SHA256SUMS          # publisher signs or publishes this
sha256sum -c SHA256SUMS                      # consumer verifies: shop-api.tar: OK
echo "malicious change" >> shop-api.tar      # someone tampers with the artifact
sha256sum -c SHA256SUMS; echo "exit=$?"
```

Expected: `shop-api.tar: OK`, then after tampering `shop-api.tar: FAILED`, `WARNING: 1 computed checksum did NOT match`, and `exit=1`. A container image **digest** and a **cosign signature** work on the same principle, with cryptographic proof of who produced the bytes (Level 4: Image Scanning and Container Security §4.14).

7. **Clean up.**

```bash
cd ~ && rm -rf ~/devsecops-demo
```

7 Real-world scenario

Illustrative. A new critical CVE is published for a popular Node.js web library. At 09:00 the security channel lights up. In the **before** world, the `shop-api` team greps repositories and asks around: three services are found by noon, a fourth (an old admin tool) at midnight. Nobody is certain they found everything.

In the **after** world, every build already generates an SBOM stored with the image. A single query against the SBOM store at 09:10 lists all 7 services using the vulnerable version, with their owners. Dependabot has already opened pull requests bumping the library; the CI pipeline runs tests, the image scan confirms the CVE is gone, and a policy at the deploy step refuses any image still containing it. By 13:00 six services are patched and the seventh is documented as not reachable and scheduled. **Lesson:** when you cannot know your exposure within minutes, you are not ready for the next big CVE; inventory and automation turn a crisis into a routine.

8 Common mistakes and best practices

Don't

- Bolt security on at the end as a gate controlled by another team.
- Fail every build on every low-severity finding; the team will learn to ignore or bypass the scanner.
- Commit secrets "temporarily". Git history is forever; a leaked key must be rotated, not just deleted from the file.
- Use `latest` tags and unpinned dependencies.
- Give CI runners long-lived, all-powerful credentials, or run untrusted pull request code with access to secrets.
- Treat scanning as a substitute for fixing: findings need owners and deadlines.

Do

- Automate the cheap checks (secret, dependency, image, IaC scans) on every pull request, and run heavier ones (DAST) on a schedule or in staging.
- Block on severity and exploitability, track the rest, and tune out false positives with documented exceptions.

- Use short-lived credentials for pipelines (for example OpenID Connect to the cloud) and least-privilege roles (Level 11: Least-privilege IAM §11.2).
- Generate an SBOM, sign artifacts, and verify signatures at deploy.
- Keep base images small and updated (Immutable Infrastructure §0.5).
- Practise: run security game days and rotate a secret on purpose to prove you can.

9 Tasks

- [ ] **T1 — Run the gate on a bad and a fixed repo** *(≈25 min, beginner)*
  - **Goal:** see a security gate fail and pass.
  - **Steps:**
    1. Complete guide steps 1-4.
    2. Save the output of the failing and passing runs.
  - **Done when:** the first run exits 1 with four `FAIL` lines and the second exits 0.
- [ ] **T2 — Add a new check** *(≈35 min, beginner)*
  - **Goal:** extend the gate with a new rule.
  - **Steps:**
    1. Add a fifth check that fails if any `.env` file is tracked in the repo directory.
    2. Add a sixth check that fails if the Dockerfile contains `curl ... | sh` or `curl ... | bash`.
    3. Create a repo that violates each and confirm each is caught.
  - **Done when:** each new violation makes the gate exit 1 with a clear message, and a clean repo still passes.
- [ ] **T3 — Install and test the pre-commit hook** *(≈25 min, intermediate)*
  - **Goal:** shift left on your own machine.
  - **Steps:**
    1. Complete guide step 5.
    2. Try `git commit --no-verify` with a bad file and note that the hook is skipped.
    3. Write down why CI must run the same gate.
  - **Done when:** a bad commit is blocked, a clean commit passes, and your note explains why local hooks are advisory.
- [ ] **T4 — Break the supply chain** *(≈30 min, intermediate)*
  - **Goal:** detect tampering and understand its limits.

- **Steps:**
  1. Complete guide step 6.
  2. Now tamper with both the artifact *and* `SHA256SUMS` (regenerate it) and run the check again.
  3. Explain why the check passes, and what you would add (signatures, a trusted channel for the checksum).
- **Done when:** you demonstrated that a checksum alone only proves integrity against accidents or a separate trusted copy, and you named signing as the fix.

- [ ] **T5 — Use a real scanner** *(≈45 min, intermediate)*
  - **Goal:** run a maintained tool instead of the toy one.
  - **Steps:**
    1. Install gitleaks or Trivy using the project's documented installation method (a package manager or a container image).
    2. Run it against your guide repo from step 1 (before the fixes).
    3. Compare its findings with the toy gate's output.
  - **Done when:** the real tool reports the AWS-style key, and you have a list of what it found that the toy gate did not (or vice versa).

- [ ] **T6 — Prove it: design a secure pipeline** *(≈50 min, advanced)*
  - **Goal:** produce a pipeline design without notes.
  - **Steps:**
    1. Draw a pipeline for `shop-api` from commit to production with every security check from section 5, placed at the stage where it is cheapest and most useful.
    2. For each check state the severity threshold that blocks the build and who owns the findings.
    3. Identify which of your pipeline's credentials would hurt most if stolen, and how you would shorten their lifetime or reduce their reach.
  - **Done when:** you have a one-page diagram and table, and can explain each placement choice and the answer to the credential question.

10 Self-check questions

1. What does "shift left" mean and why does it save money?
2. Name four automated security checks and what each one looks at.
3. What is an SBOM and when did it matter in the Log4Shell response?

4. Why is a digest safer than a tag?
5. Why can a pre-commit hook not be your only control?

ANSWERS

1. Moving security checks earlier in the lifecycle (to the developer's machine and pull request). A flaw is cheapest to fix when the code is fresh and nothing is deployed.
2. Secret scanning (committed credentials), SAST (your source code), SCA or dependency scanning (known CVEs in libraries), image scanning (OS and library CVEs in a container), IaC scanning (insecure infrastructure config), DAST (the running app).
3. A software bill of materials, a machine-readable inventory of the components in an artifact. It let organisations find every service using the vulnerable Log4j quickly instead of searching manually.
4. A tag is a movable label; a digest is the hash of exact content, so it always identifies the same bytes and any change is detectable.
5. Developers can bypass or fail to install local hooks. The same checks must run in CI, which is enforced by the server and gates merges.

11 Further reading

- OWASP resources: the OWASP Top 10 and OWASP DevSecOps Guideline (owasp.org).
- SLSA framework (slsa.dev) and Sigstore documentation (sigstore.dev).
- NIST Secure Software Development Framework (SSDF), SP 800-218 (csrc.nist.gov).
- Trivy documentation (trivy.dev) and gitleaks repository (github.com/gitleaks/gitleaks).
- GitHub documentation on Dependabot, secret scanning and code scanning (docs.github.com).
- See also: Level 11: OWASP Top 10 §11.5 and Level 11: Container Image Scanning §11.9.